

# Class 6: R functions

Shuk Yee Leung (PID: A18050465)

## Table of contents

|                                        |   |
|----------------------------------------|---|
| 1. Function basics . . . . .           | 1 |
| 2. Generate DNA sequence . . . . .     | 2 |
| 3. Generate Protein function . . . . . | 4 |

## 1. Function basics

Let's start writing our first silly function to add some numbers:

Every R function has 3 things:

- name (we got to pick this)
- input arguments (there can loads of these sperated by a comma)
- the body (the R code that does the work)

```
add <- function(x, y=10, z=0){  
  x + y + z  
}
```

I can just use this function like any other function as long as R knows about it (i.e. run the code chunk)

```
add(1, 100)
```

```
[1] 101
```

```
add( x=c(1,2,3,4), y=100)
```

```
[1] 101 102 103 104
```

```
add(1)
```

```
[1] 11
```

Functions can have “required” input arguments and “optional” input arguments. The optional arguments are defined with an equals default value (`y=10`) in the function definition.

```
add(x=1, y=100, z=10)
```

```
[1] 111
```

Q. Write a function to return a DNA sequence of a user specified length? Call it `genetate_dna()`

The `sample()` function can help here

```
#generate_dna <- function(size=5) { }  
  
students <- c("jeff", "jeremy", "peter")  
  
sample(students, size=5, replace=TRUE)
```

```
[1] "jeremy" "jeff"   "jeremy" "jeff"   "peter"
```

## 2. Generate DNA sequence

Now work with `bases` rather than `students`

```
bases <- c("A", "C", "G", "T")  
sample(bases, size=10, replace = TRUE)
```

```
[1] "T" "G" "A" "C" "T" "T" "A" "A" "G" "A"
```

Now I have a working ‘snippet’ of code I can use this as the body of my first function here:

```
generate_dna <- function(size=5) {  
  bases <- c("A", "C", "G", "T")  
  sample(bases, size=size, replace = TRUE)  
}
```

```
generate_dna(100)
```

```
[1] "A" "C" "A" "A" "G" "T" "T" "T" "C" "T" "A" "T" "C" "G" "C" "G" "T" "A"  
[19] "C" "G" "A" "G" "T" "A" "G" "A" "A" "G" "G" "G" "A" "T" "G" "C" "A" "A"  
[37] "A" "G" "A" "T" "A" "A" "T" "G" "C" "C" "G" "A" "C" "G" "T" "T" "T" "T"  
[55] "G" "C" "T" "C" "G" "G" "C" "A" "A" "C" "C" "A" "A" "G" "T" "T" "A" "G"  
[73] "A" "G" "T" "A" "G" "C" "G" "C" "G" "T" "G" "G" "C" "C" "G" "G" "T" "T"  
[91] "C" "A" "G" "T" "T" "A" "G" "T" "C" "G"
```

```
generate_dna()
```

```
[1] "G" "T" "A" "G" "A"
```

I want the ability to return a sequence like “AGTACCTG” i.e. a one element vector where the bases are all together.

```
generate_dna <- function(size=5, together=TRUE) {  
  bases <- c("A", "C", "G", "T")  
  sequence <- sample(bases, size=size, replace = TRUE)  
  
  if(together) {  
    sequence <- paste(sequence, collapse = "")  
  }  
  return(sequence)  
}
```

```
generate_dna()
```

```
[1] "ATGTC"
```

```
generate_dna(together = FALSE)
```

```
[1] "A" "T" "T" "A" "A"
```

### 3. Generate Protein function

Q. Write a protein sequence generating function that will return sequences of a user specified length?

We can get the set of 20 natural amino-acids from the **bio3d** package.

```
aa <- bio3d::aa.table$aa1[1:20]

generate_protein <- function(size=5, together=TRUE) {

  ## Get the 20 amino-acids as a vector
  aa <- bio3d::aa.table$aa1[1:20]
  sequence <- sample(aa, size, replace=TRUE)

  ## Optionally return a single element string
  if(together){
    sequence <- paste(sequence, collapse="")
  }
  return(sequence)
}
```

Q. Generate random protein sequences of length 6 to 12 amino acids.

```
generate_protein(7)
```

```
[1] "EDWRLEH"
```

```
generate_protein(8)
```

```
[1] "LQHWMEVY"
```

```
generate_protein(9)
```

```
[1] "GSINWRNFH"
```

```
# generate_protein(size=6:12)
```

We can fix this inability to generate multiple sequences by either editing and adding to the function body code (e.g. a for loop) or by using the R **apply** family of utility functions.

```
sapply(6:12, generate_protein)
```

```
[1] "WVPYLF"      "YGATLIT"      "EGMNRTIK"      "QYDGAVQHC"      "SHFVYRWVTR"  
[6] "LFQVMYNIYWQ" "VWHLNKHVAMSK"
```

It would cool and useful if I could get FASTA format output.

```
ans <- sapply(6:12, generate_protein)  
ans
```

```
[1] "CPGEDS"      "FTLTRLW"      "YHKQREGV"      "AEIMYGQQF"      "HVTGWHLWV"  
[6] "IHQKQVPDAQE" "PGNRMNPSMVRS"
```

```
cat(ans, sep="\n")
```

```
CPGEDS  
FTLTRLW  
YHKQREGV  
AEIMYGQQF  
HVTGWHLWV  
IHQKQVPDAQE  
PGNRMNPSMVRS
```

I want this to look like FASTA format with an ID line. e.g.

```
>ID.6  
QRDMAD  
>ID.7  
YINTMSA  
>ID.8  
PQPMWVSD
```

The functions `paste()` and `cat()` can help us here...

```
cat( paste(">ID.", 7:12, "\n", ans, sep=""), sep="\n" )
```

```

>ID.7
CPGEDS
>ID.8
FTLTRLW
>ID.9
YHKQREGV
>ID.10
AEIMYGQQF
>ID.11
HVTGWHLWV
>ID.12
IHQKQVPDAQE
>ID.7
PGNRMNPSMVR

```

```

id.line <- paste(">ID.", 6:12, sep="")
id.line

```

```
[1] ">ID.6" ">ID.7" ">ID.8" ">ID.9" ">ID.10" ">ID.11" ">ID.12"
```

```

id.line <- paste(">ID.", 6:12, sep="")
seq.line <- paste(id.line, ans, sep="\n")
cat(seq.line, sep="\n", file="myseq.fa")

```

Q. Determine if these sequences can be found in nature or are they unique? Why or why not?

I BLASTp searched my FASTA format sequences against NR and found that length 6 and 7 are not unique and can be found in the databases with 100% coverage and 100% identity. Length 8 is unique 87.50% identity.

Random sequences of length 8 and above are unique and can't be found in the databases.